

Multivariate Monte Carlo Simulation

Portfolio VaR Engine — Model Documentation

Risk Modelling — Trade Republic

April 2026

1 Overview

The Monte Carlo (MC) engine generates a large number of independent synthetic return paths for all assets in the portfolio simultaneously, using a multivariate distribution calibrated to historical data. VaR and CVaR are then estimated empirically from the simulated portfolio P&L distribution. This is the only engine that natively captures cross-asset dependence via a full covariance matrix.

Engine key: 'mc'

Sources:

- `risk_models/mc_models.py`, lines 16–96 (`mc_simulate`)
- `risk_analytics/risk_engines.py`, lines 154–186 (VaR/CVaR aggregation)
- `risk_models/support_models.py`, lines 49–61 (`ewma_covariance`)

2 Covariance Matrix Estimation

Three options are available for the input covariance matrix $\hat{\Sigma} \in \mathbb{R}^{N \times N}$ (where N is the number of assets):

2.1 EWMA Covariance (default: `ewma_cov=True`)

Following RiskMetrics, the EWMA covariance matrix over a window of T observations is:

$$\hat{\Sigma}_{\text{EWMA}} = \frac{\sum_{k=0}^{T-1} \lambda^k (\mathbf{r}_{T-k} - \bar{\mathbf{r}})(\mathbf{r}_{T-k} - \bar{\mathbf{r}})^\top}{\sum_{k=0}^{T-1} \lambda^k} \quad (1)$$

where $\bar{\mathbf{r}}$ is the sample mean vector and $\lambda = 0.94$ is the decay factor. More recent observations receive exponentially higher weight.

2.2 Sample (Historical) Covariance (`ewma_cov=False`)

$$\hat{\Sigma}_{\text{hist}} = \frac{1}{T-1} \sum_{t=1}^T (\mathbf{r}_t - \bar{\mathbf{r}})(\mathbf{r}_t - \bar{\mathbf{r}})^\top \quad (2)$$

2.3 Maximum Likelihood Estimator (`mlest_cov=True`)

Uses `sklearn.covariance.EmpiricalCovariance` on 500 Monte Carlo samples drawn from a multivariate Normal with the historical covariance. This option is available for stress scenarios where `stress_means` are specified.

3 Simulation Procedure

3.1 Drawing Simulated Paths

For each of n simulation runs $i = 1, \dots, n$, draw a matrix of T_{sim} multivariate daily returns:

Multivariate Normal:

$$\mathbf{r}^{(i)} \stackrel{i.i.d.}{\sim} \mathcal{N}_N(\boldsymbol{\mu}, \hat{\Sigma}), \quad \mathbf{r}^{(i)} \in \mathbb{R}^{T_{\text{sim}} \times N} \quad (3)$$

Multivariate Student- t :

$$\mathbf{r}^{(i)} \stackrel{i.i.d.}{\sim} t_N(\boldsymbol{\mu}, \hat{\Sigma}, \nu), \quad \mathbf{r}^{(i)} \in \mathbb{R}^{T_{\text{sim}} \times N} \quad (4)$$

where $\boldsymbol{\mu}$ is the historical mean vector and ν is the degrees-of-freedom parameter (default $\nu = 5$).

The Student- t option produces heavier-tailed joint distributions and is therefore more conservative for tail risk estimation.

3.2 Portfolio Return Paths

For each simulation i , the portfolio return path is:

$$r_{pf,t}^{(i)} = \mathbf{w}^\top \mathbf{r}_t^{(i)}, \quad t = 1, \dots, T_{\text{sim}} \quad (5)$$

giving a simulated P&L vector $\mathbf{r}_{pf}^{(i)} \in \mathbb{R}^{T_{\text{sim}}}$ for each simulation run.

4 VaR and CVaR Estimation

4.1 Value-at-Risk

For each simulation run i , compute the empirical α -quantile of the portfolio return path:

$$\text{VaR}_\alpha^{(i)} = -Q_\alpha(\mathbf{r}_{pf}^{(i)}) \quad (6)$$

The reported VaR is the **median** across all n simulation runs:

$$\widehat{\text{VaR}}_\alpha = \text{median}_{i=1, \dots, n} \left\{ \text{VaR}_\alpha^{(i)} \right\} \quad (7)$$

Using the median (rather than the mean) provides robustness against extreme outlier simulations.

4.2 Conditional Value-at-Risk (Expected Shortfall)

For each simulation run i , CVaR is the mean of returns that fall at or below the α -quantile:

$$\text{CVaR}_\alpha^{(i)} = -\frac{1}{|\mathcal{S}_\alpha^{(i)}|} \sum_{t \in \mathcal{S}_\alpha^{(i)}} r_{pf,t}^{(i)}, \quad \mathcal{S}_\alpha^{(i)} = \left\{ t : r_{pf,t}^{(i)} \leq Q_\alpha(\mathbf{r}_{pf}^{(i)}) \right\} \quad (8)$$

The reported CVaR is again the median across simulations:

$$\widehat{\text{CVaR}}_\alpha = \text{median}_{i=1, \dots, n} \left\{ \text{CVaR}_\alpha^{(i)} \right\} \quad (9)$$

4.3 Holding-Period Scaling

$$\widehat{\text{VaR}}_\alpha^{(H)} = \widehat{\text{VaR}}_\alpha^{(1)} \cdot \sqrt{H}, \quad \widehat{\text{CVaR}}_\alpha^{(H)} = \widehat{\text{CVaR}}_\alpha^{(1)} \cdot \sqrt{H} \quad (10)$$

5 Stress Testing Extension

The `stress_means` parameter allows replacing the historical mean vector $\boldsymbol{\mu}$ with a user-specified stress scenario mean vector $\boldsymbol{\mu}^{\text{stress}}$, while keeping the covariance structure unchanged:

$$\mathbf{r}^{(i)} \sim \mathcal{N}_N(\boldsymbol{\mu}^{\text{stress}}, \hat{\Sigma}) \quad (11)$$

6 Parameters

Parameter	Type	Default	Description
<code>window</code>	int	250	Lookback window for covariance estimation
<code>n_sim</code>	int	1000	Number of Monte Carlo simulation runs
<code>distr</code>	str	'norm'	Distribution: 'norm' or 't'
<code>decay</code>	float	0.94	EWMA decay λ for covariance matrix
<code>ewma_cov</code>	bool	True	Use EWMA covariance (else: sample covariance)
<code>qtls</code>	list	[0.05, 0.01, 0.001]	Tail probability levels
<code>holding_period</code>	int	1	Forecast horizon in days
<code>verbose</code>	bool	True	Print progress information

Additional parameters passed directly to `mc_simulate()`:

Parameter	Type	Default	Description
<code>deg_f</code>	int	5	Student- t degrees of freedom
<code>mlest_cov</code>	bool	False	Use ML covariance estimator
<code>stress_means</code>	list	[]	Stress scenario mean vector

7 Advantages and Limitations

Advantages	Limitations
Captures cross-asset dependencies	Computationally intensive ($n \times T_{\text{sim}}$ draws)
No parametric tail assumption required	Does not capture volatility clustering
Flexible: Normal or Student- t	Covariance matrix may be poorly conditioned
Stress scenario support	Convergence requires large n for extreme quantiles
Transparent and intuitive	Square-root scaling is approximate

8 Convergence Guidance

For stable 99.9% VaR estimates ($\alpha = 0.001$), at least $n = 5,000$ simulations are recommended. With the default $n = 1,000$, the 99.9% estimate has a coefficient of variation of approximately 10%.

9 Usage Example

```
from risk_analytics import risk_engines as re

var_mc = re.portfolio_var(
    rets_orig = rets,
    wgts      = weights,
```

```
engine      = 'mc',
fmt_engine = {
    'window'      : 250,
    'n_sim'       : 5000,
    'distr'       : 'norm',
    'decay'       : 0.94,
    'ewma_cov'    : True,
    'qtls'        : [0.05, 0.01, 0.001],
    'holding_period' : 1,
    'verbose'     : True,
}
)
```

10 References

J.P. Morgan/Reuters (1996). *RiskMetrics Technical Document*, 4th ed.
Jorion, P. (2006). *Value at Risk*, 3rd ed. McGraw-Hill.